University of Florence

School of Engineering - Department of Information Engineering

Thesis of the Master's Degree in Computer Engineering

Hybrid Quantum-Classical Architecture for Solving Large-Scale QUBO Problems: A Divide et Impera Approach

Candidate
Giacomo Orsucci

Supervisors
Prof. Benedetta Picano
Prof. Romano Fantacci

Co-supervisor
Dr. Claudio Cicconetti

Academic Year 2025/2026

Contents

Chapter 1

Introduction to Quantum Computing

In recent years, new and promising computing paradigms and architectures have emerged, complementing (and possibly competing with or outperforming in the near future) classical computing to solve highly complex problems. Among them, quantum computing stands out as one of the most promising and disruptive paradigms, expected to fundamentally revolutionize the way we process information.

This paradigm is profoundly different from classical computing, which uses a binary system of bits (0 or 1) to represent and process data; quantum computing harnesses qubits, a completely different representation of information. Through quantum superposition, a qubit can exist simultaneously in a combination of states $|0\rangle$ and $|1\rangle$. Furthermore, quantum entanglement allows a deep correlation between qubits even over large distances, enabling the system to represent and process information in a fundamentally different way, with the potential to outperform classical computing in specific tasks [1, 2].

In this chapter, we will provide a brief overview of quantum computing, its evolution, and its promises. We will also compare different hardware paradigms, focusing on superconductors and neutral atoms, and discuss the characteristics and geometric constraints of neutral atom platforms.

1.1 Promises of quantum computing

The quantum computing paradigm currently promises to revolutionize information processing, offering the potential to solve problems that are intractable for classical computers. It is thus immediate to consider all the potential applications and fields that are currently facing the computational limits of NP-hard problems. While many heuristics, approximation algorithms, and metaheuristics have been developed to solve these problems, they are not always able to provide optimal solutions. Consequently, it is straightforward to recognize the potential of quantum computing in fields such as cryptography, optimization, machine learning, and drug discovery [3], where the ability to process large datasets and solve NP-hard problems on larger scales can lead to significant breakthroughs.

As described in [4], which presents an overview of practical applications for neutral atom quantum architectures and discusses the primary characteristics and potential use cases of this technology, the most explored application fields for this type of platform are MIS, Max-Cut, Max-Clique, QUBO problems and similar. Additionally, a complete panorama of this paradigm, including its challenges, is detailed in [1]. Thus, to provide a summarized but complete introduction in agreement with the recent literature, we can conclude that quantum computing is a disruptive paradigm offering new opportunities for solving complex problems in various fields. However, it is

still in its early stages of development, and many challenges remain to be addressed before it can reach its full potential.

In addition, it is very important to emphasize that these limitations and challenges must not be seen as a reason to avoid exploring this paradigm, but rather as a motivation to understand its boundaries and develop new algorithms capable of leveraging its unique capabilities. The importance of this aspect is highlighted by the sector's market capitalization, estimated at approximately USD 10.13 billion in 2022 and the high expectations placed upon it with a projected and estimated market capitalization of USD 125 billion by 2030 [1].

In the following sections, the main characteristics of neutral atom quantum computing are explored in more detail, together with their physical constraints, to provide a clear sense of what we are meant to experiment and explore, why with these modalities and why some specific limitations will be contemplated and expected in the next chapters of this thesis.

1.2 Hardware paradigms compared: Superconductors vs. Neutral Atoms

Currently, the quantum computing landscape is characterized by rapid technological advancement and intense competition. A diverse ecosystem of tech giants, research institutions, and startups is actively investigating various physical architectures. The shared objective is to develop a highly efficient and scalable computing paradigm capable of overcoming the fundamental challenges of the field, primarily quantum noise, limited coherence times, and hardware scalability [1].

To illustrate the diversity of this ecosystem, it is sufficient to look at the

different technological pathways chosen by major industry players. Companies such as Google [5], Microsoft [6], and D-Wave [7] have heavily invested in superconducting qubits, whereas startups like Pasqal [8] and QuEra Computing [9] are pioneering platforms based on neutral atoms. Simultaneously, other organizations are exploring alternative technologies, including trapped ions, photonic circuits, silicon spin qubits, and innovative quantum error correction approaches, such as the hardware-efficient *cat qubits* developed by Alice & Bob [10], which aim to intrinsically mitigate decoherence and pave the way toward fault-tolerant architectures.

This section provides a comparative analysis of the two most prominent paradigms relevant to this work: superconducting circuits and neutral atoms.

1.2.1 Superconducting qubits

Without delving deeply into the physical details (which are beyond the scope of this thesis), superconducting qubits are based on Josephson junctions, superconducting circuits that exhibit quantum behavior at temperatures near absolute zero. These qubits are typically fabricated using advanced lithographic techniques and can be integrated into complex solid-state circuits. Superconducting qubits offer the advantage of very fast gate operations and relatively high fidelity, but they require extreme cryogenic cooling to maintain their quantum state [5].

A comprehensive overview of superconducting qubit technologies and the major players leading the field can be found in [3]. Currently, this type of qubit is the most mature and widely adopted in the industry; nevertheless, it faces significant challenges regarding scalability and coherence times. The strict requirement for cryogenic cooling and the engineering complexity of scaling up the number of qubits while maintaining high fidelity and mitigat-

ing cross-talk remain significant obstacles to achieving practical, large-scale quantum computing [3].

D-Wave is among the most well-known companies in this space, having developed commercial quantum annealers based on superconducting qubits designed specifically to solve optimization problems [7]. It is important to note that D-Wave's approach differs from the universal gate-based model pursued by Google and IBM, as it is tailored for specific optimization landscapes rather than general-purpose computing. However, for Quadratic Unconstrained Binary Optimization (QUBO) problems (which represent the core mathematical framework investigated in this thesis) quantum annealing remains one of the most effective approaches. While D-Wave's specific platform will not be extensively discussed in the remainder of this work, it stands as a commercially viable benchmark for solving large QUBO instances through purely quantum or hybrid approaches.

1.2.2 Neutral Atom platforms

Neutral atom platforms, on the other hand, utilize individual atoms trapped in optical lattices or optical tweezers to act as qubits. These atoms can be manipulated using finely tuned laser light, allowing their quantum states to be controlled with extreme precision. A primary advantage of neutral atom systems is their excellent isolation from the environment, which translates into long coherence times. Furthermore, they offer significant scalability potential, as large arrays of atoms can be trapped, dynamically positioned, and manipulated simultaneously. However, they also face significant engineering challenges, particularly regarding the complexity of controlling massive optical arrays and maintaining atomic stability within the traps over extended periods [3, 8].

These platforms are exceptionally promising for quantum simulation and combinatorial optimization tasks. By exciting the atoms to highly energetic states, they can naturally implement strong, long-range interactions that are difficult to achieve natively with other qubit technologies [8]. Due to this inherent physical capability, neutral atom architectures represent the primary focus of this thesis.

Among the main players in the field, Pasqal and QuEra Computing are notable for their pioneering work in developing these platforms. These companies have made significant strides in demonstrating the feasibility of large-scale neutral atom systems and their potential applicability in solving complex optimization problems [3, 8, 9].

Specifically, this work explores a hybrid quantum-classical architecture designed to tackle large-scale QUBO problems using these precise devices. Consequently, their capabilities and constraints are extensively investigated and discussed throughout the following chapters. As a foundation, the next section details the main characteristics and geometric constraints inherent to neutral atom platforms, which act as crucial parameters for the design of the hybrid architecture proposed in this thesis.

1.3 Characteristics of Neutral Atom platforms

This section focuses on the main characteristics and geometric constraints of neutral atom platforms, which are crucial for understanding the limitations and capabilities of these systems in the context of solving large-scale QUBO problems.

Neutral atom arrays A neutral atom array can be conceptualized as a hardware register where each individual atom acts as a qubit. These atoms

are arranged in 1D, 2D (as in the context of this work), or 3D configurations [8, 11], as illustrated in Figure 1.1.

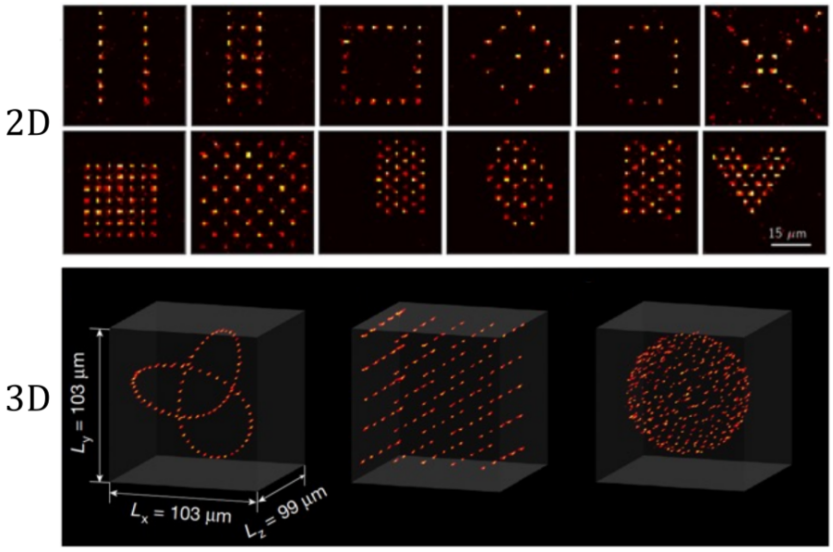

Figure 1.1: 2D and 3D neutral atom arrays. Image extracted from [8].

Unlike superconducting systems, where qubits are statically fabricated on a physical chip, neutral atoms can be dynamically arranged and manipulated using optical tweezers. This inherent flexibility allows for the creation of custom geometries and the implementation of specific interaction topologies between qubits, which is highly advantageous for solving optimization problems that demand tailored connectivity.

However, because the register is not permanently fixed, it must be reconstructed for each computation cycle. The overall execution process consists of three main phases:

- Register preparation
- Quantum processing
- Register readout

The primary focus of this thesis, however, precedes the physical execution phase. It centers on the algorithmic challenge of efficiently embedding a QUBO problem onto this spatially constrained register to find optimal or near-optimal solutions. While the remainder of this work focuses mainly on the algorithmic approach, leveraging libraries and remote simulation infrastructures rather than delving into low-level hardware control, understanding the physical constraints of the QPU is essential. Therefore, following the explanations provided by [8], we briefly outline the physical mechanisms behind register loading and readout.

Register loading Configuring the atomic register requires a complex optical setup, relying on arrays of optical tweezers alongside the components illustrated in Figure 1.2.

In essence, the process begins inside a vacuum chamber, where a laser system cools and confines a cloud of atoms. Subsequently, high-precision lenses focus light beams into extremely tight spots, creating optical tweezers. Each optical trap is designed to isolate and hold a single atom. By leveraging holographic techniques through a Spatial Light Modulator (SLM), it is possible to project these tweezers into predefined positions, generating the desired geometric array. This mechanism is the cornerstone of the technology's scalability: theoretically, scaling the system primarily requires increasing laser power and improving optical resolution to project more traps.

However, loading atoms from the cooled cloud into the traps is a stochastic process. Due to light-assisted collisions, each optical tweezer has roughly a 50% probability of capturing exactly one atom, leaving the remaining traps empty. To construct a defect-free computational register, the system must perform an active rearrangement. This is achieved using dynamic tweezers

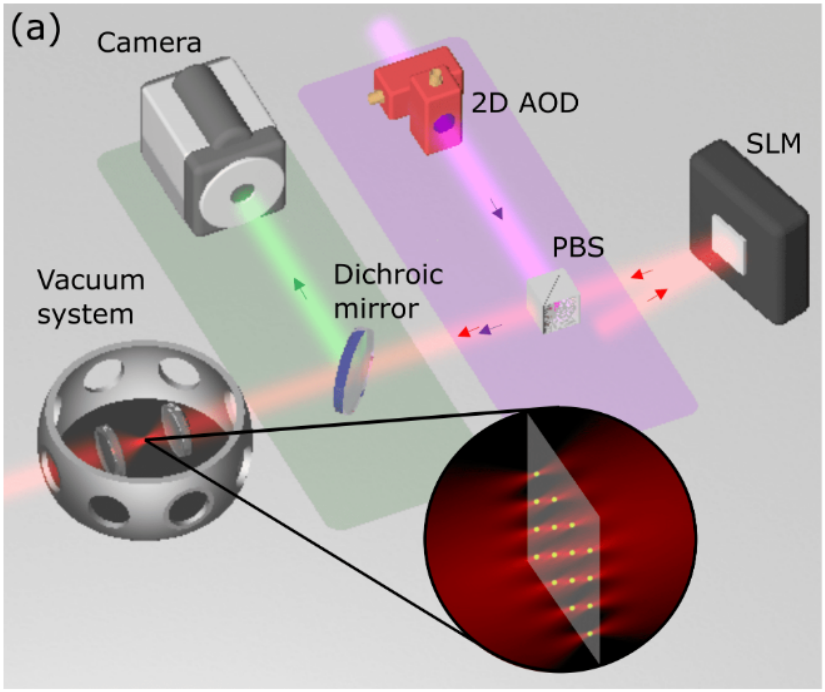

Figure 1.2: Main hardware components of a neutral atom QPU. The trapping laser light (red) is shaped by a Spatial Light Modulator (SLM). Moving tweezers (purple), used to rearrange atoms, are controlled by a 2D Acousto-Optic Deflector (AOD) and superimposed using a Polarizing Beam-Splitter (PBS). The green fluorescence emitted by the atoms is separated by a dichroic mirror and captured by a camera during the readout phase. Image adapted from [8].

governed by Acousto-Optic Deflectors (AODs), which sort and move atoms from filled traps into empty ones. This sorting process is repeated until the target array is completely filled, resulting in a perfect register ready for the quantum processing phase [8].

Quantum processing and register readout Once the atomic array is perfectly assembled, the quantum processing phase can begin. It is worth noting a significant timescale disparity in this hardware: while the actual quantum computation (performed via precisely timed laser pulses) is extremely fast, typically lasting less than 100 µs, the entire computational cycle is dominated by the physical loading and readout phases, requiring approximately 200 ms in total [8].

After the algorithm is executed, the final state of the register must be measured to extract the classical solution. This readout phase is performed through fluorescence imaging. The physical system is tuned so that atoms remaining in the ground state (representing the logical qubit state $|0\rangle$) emit photons and appear as bright spots on the image. Conversely, atoms excited to the Rydberg state (representing the logical qubit state $|1\rangle$) do not fluoresce and remain dark.

These optical signals are captured by highly sensitive Electron-Multiplying CCD (EMCCD) cameras. This detection method is extremely reliable, reaching readout efficiencies above 98.6%, a performance highly competitive with the readout fidelities typical of superconducting circuits [8]. Finally, due to the inherently probabilistic nature of quantum mechanics, a single execution yields only one possible classical bit-string. Therefore, the complete sequence of preparation, processing, and readout must be repeated hundreds or thousands of times (taking multiple *shots*) to reconstruct the statistical

probability distribution of the outcomes and successfully identify the optimal solution.

Quantum analog mode execution Neutral atom platforms can operate in both digital and analog modes. This work concentrates exclusively on the latter; please refer to [8] for more information on the former. The dynamics of the analog mode are well-described by the Ising model, governed by the following Hamiltonian:

$$\hat{\mathcal{H}}(t) = \hbar \sum_{i=1}^{N} \left(\frac{\Omega(t)}{2} \hat{\sigma}_i^x - \delta(t) \hat{n}_i \right) + \sum_{i<j} \frac{C_6}{|r_i - r_j|^6} \hat{n}_i \hat{n}_j \tag{1.1}$$

This equation can be conceptually divided into two parts. The first summation dictates the evolution of single atoms driven by the external laser field, while the second summation governs the interactions between pairs of atoms. Specifically, the parameters and operators are defined as follows:

- N is the total number of atoms (qubits) in the register;
- $\Omega(t)$ is the time-dependent Rabi frequency, representing the amplitude of the laser exciting the atoms;
- $\delta(t)$ is the detuning parameter of the laser frequency;
- $\hat{\sigma}_i^x$ is the Pauli-X operator acting on atom i, which drives the transition between the ground state $|0\rangle$ and the Rydberg state $|1\rangle$;
- $\hat{n}_i$ is the occupation number operator (defined as $|1\rangle_i\langle 1|_i$) which evaluates to 1 if atom i is in the Rydberg state, and 0 otherwise;
- r_i is the spatial position of atom i;
- C_6 is the Van der Waals interaction coefficient, which is a constant dependent on the specific atomic species and platform used [12].

Crucially, the interaction term (represented by the second summation) in the shown Hamiltonian accounts for the interaction between individual spins, in other words it accounts for the energy penalty incurred when two spatially close qubits occupy the Rydberg state simultaneously. When excited, qubits transition to the Rydberg state; however, if they are physically in close proximity, they experience the *Rydberg blockade effect*. In this regime, maintaining the Rydberg state by the first qubit heavily suppresses the excitation of the second one. Qubits in Rydberg states interact with each other proportionally to this interaction term, following the Van der Waals potential for dipole-dipole interactions:

$$V_{ij} = \frac{C_6}{r_{ij}^6} \tag{1.2}$$

where:

- V_{ij} is the interaction energy between atoms i and j;
- C_6 is the Van der Waals coefficient, which depends on the atom type and scales massively with the excitation level;
- r_{ij} is the spatial distance between the two atoms [8].

Chapter 2

The Embedding Problem: Theoretical Framework

This chapter introduces the theoretical framework behind the embedding problem for Neutral Atom Quantum Architectures. The aim of this part is to explain the complexity of finding efficient embeddings, starting with an introduction of basic concepts of computational complexity and graph theory, and then proceed to define Unit Disk Graphs (UDG) and the Maximum Independent Set (MIS) problem. Finally, these concepts will be connected to the current State of the Art (SOTA) and the Quadratic Unconstrained Binary Optimization (QUBO) formulation.

2.1 Combinatorial Optimization and Computational and Geometrical Complexity

Combinatorial optimization is a prominent branch of applied mathematics and computer science that focuses on finding an optimal object from a finite, discrete set of possible solutions. Specifically, the objective is to identify a

solution that minimizes (or maximizes) a well-defined cost function, often subject to a set of strict mathematical constraints [13].

These problems commonly involve discrete mathematical structures such as graphs, permutations, or boolean variables and are ubiquitous in fields ranging from logistics and network routing to machine learning and quantum physics [14]. Although the solution space for these problems can be strictly finite, the number of possible configurations still can be overwhelmingly large.

Moreover, combinatorial optimization frequently encounters NP-Hard problems, where finding the exact global optimum becomes computationally intractable. Consequently, the field heavily leverages heuristics and metaheuristics to find high-quality approximate solutions within a reasonable timeframe. Within this computationally demanding landscape, Quantum Computing emerges as a disruptive paradigm; by exploiting quantum mechanical phenomena, it promises to solve significantly larger NP-Hard instances and achieve higher-quality approximate solutions more efficiently than classical approaches [3].

Provided this general computational complexity, a secondary, equally formidable challenge arises when utilizing neutral atom platforms: the geometrical complexity. As introduced in Chapter 1, executing an embedding algorithm requires mapping (or *embedding*, precisely) the logical variables of the optimization problem onto the physical atoms arranged in a 2D or 3D register [8]. In our specific case, focusing on a 2D plane, the hardware imposes strict geometric constraints and specific interaction boundaries governed by the Ising model and the Rydberg blockade. Finding an efficient spatial mapping that preserves the problem's theoretical connectivity while respecting these strict physical limitations is, remarkably, an NP-Hard problem in itself [11].

2.2 Unit Disk Graphs (UDG) and the Maximum Independent Set (MIS)

A graph can be formally denoted as:

$$G = (V, E) \tag{2.1}$$

where V is the set of vertices (or nodes) and E is the set of edges connecting them. Graphs are very common mathematical structures used to model relations between objects, and their properties are supported by a vast, historically established and robust theoretical foundation [15].

Thus, while an exhaustive review of graph theory is beyond the scope of this work, these structures will be briefly presented because of fundamental interest. Indeed, a significant portion of current literature focuses on graph-based combinatorial problems, primarily because graphs serve as the standard mathematical interface to encode and map optimization tasks onto quantum architectures. Prominent examples of such NP-Hard problems include the Maximum Cut (Max-Cut), the Traveling Salesperson Problem (TSP), and the Maximum Independent Set (MIS), which have extensively served as standard benchmarks for evaluating the performance of quantum algorithms [16].

To specialize our focus from general graphs, we introduce the concept of Unit Disk Graphs (UDG). As formalized by Clark et al. [17], UDGs can be defined through three mathematically equivalent geometric models:

- **Distance Model:** Given a set of n points in the Euclidean plane, a n-vertex graph is formed where each vertex corresponds to a point ($|V| = n$). An edge exists between two vertices if and only if the Euclidean distance $d(u, v)$ between the two corresponding points u and

v is at most a specified threshold radius r. Mathematically, the edge set is defined as:

$$E = \{\{u, v\} \mid u, v \in V,\ d(u, v) \leq r\} \tag{2.2}$$

- **Intersection Model:** Consider a set of n equal-sized circles in the plane. The UDG is the intersection graph of these circles: each vertex corresponds to a circle, and an edge appears between two vertices if their corresponding circles overlap (tangent circles are also assumed to intersect).
- **Containment Model:** Alternatively, considering again n equal-sized circles, an edge exists between two vertices if the circle corresponding to one vertex geometrically contains the center of the other.

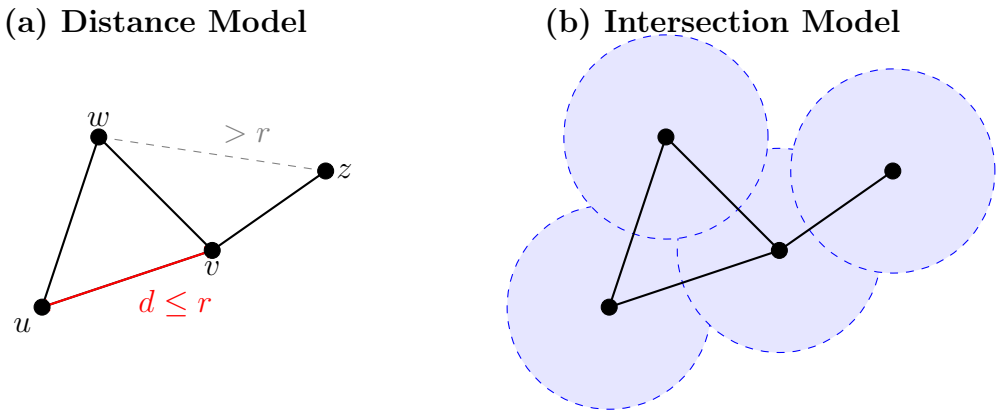

Figure 2.1: The equivalent geometric representations of a Unit Disk Graph (UDG) cited from [17]. **(a)** The Distance Model connects nodes u and v if their Euclidean distance is $d \leq r$. **(b)** The Intersection Model places a disk of radius $r/2$ centered on each node; edges are formed exclusively where disks overlap.

Given any of these three definitions, the other two can be derived in linear time, demonstrating that they describe the exact same class of graphs. Historically, UDGs have proven to be an excellent mathematical framework

for modeling broadcast and cellular networks, effectively resolving real-world problems such as frequency assignment and emergency signal routing [17].

In the context of this thesis, the distance model is "the elected" model: replacing the generic threshold distance r with the Rydberg radius (R_b) perfectly captures the physical interaction connectivity of neutral atoms in a quantum register.

As previously mentioned, the Maximum Independent Set (MIS) is a classic NP-Hard combinatorial optimization problem. While canonically formulated on general graphs, it holds massive relevance for our architecture because the problem famously remains NP-Hard even when restricted to Unit Disk Graphs [17].

Formally, given a graph $G = (V, E)$, an *Independent Set* (IS) is defined as a subset of vertices $I \subseteq V$ such that no two vertices within I are adjacent. Mathematically, this constraint translates to:

$$\forall u, v \in I, \quad \{u, v\} \notin E \tag{2.3}$$

The MIS problem entails finding an independent set that contains the largest possible number of vertices; in other words, the objective is to maximize the cardinality $|I|$.

Given the aforementioned definitions, a straightforward and perfect isomorphism emerges between the abstract MIS problem on a UDG and the physical dynamics of neutral atom arrays. Specifically, the geometric threshold radius r of the UDG corresponds directly to the Rydberg blockade radius (R_b).

Consequently, the core mathematical constraint of the MIS problem, which strictly forbids selecting two adjacent vertices, is natively and physically enforced by the Rydberg blockade phenomenon, which prevents any two atoms within a distance R_b from being simultaneously excited to the Ryd-

berg state. Because of this perfect theoretical alignment, applying a global laser pulse designed to maximize the number of excited atoms in the 2D register forces the quantum many-body system to naturally evolve toward a ground state that inherently encodes the exact solution to the Maximum Independent Set of the corresponding geometric graph [8].

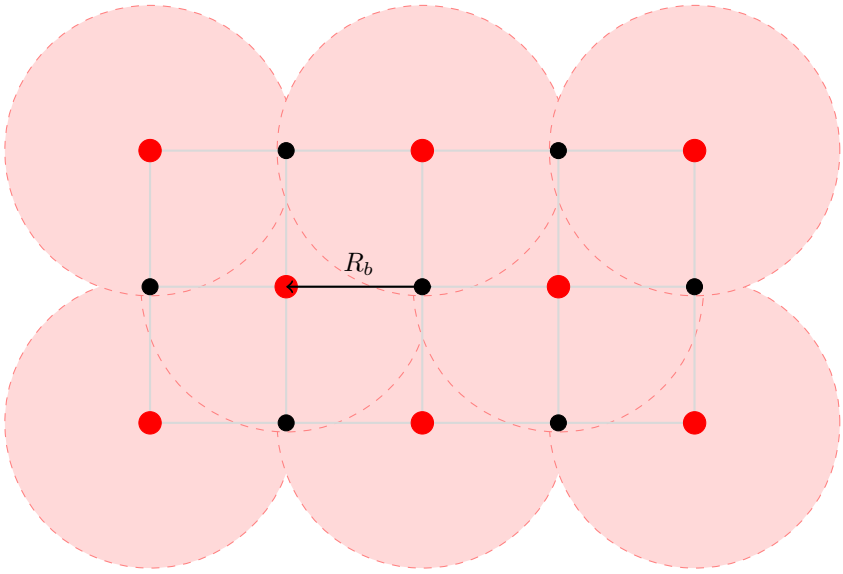

Figure 2.2: Physical realization of the Maximum Independent Set (MIS) via the Rydberg blockade. Red dots represent atoms excited to the Rydberg state (**MIS selected nodes**), while black dots are atoms remained in the ground state (**unselected nodes**). The shaded red regions represent the Rydberg blockade volume (radius R_b). The physical mechanism preventing two Rydberg excitations from overlapping perfectly enforces the mathematical constraint of the MIS problem.

2.3 Literature and State of the Art

Defining a definitive State of the Art (SOTA) for the embedding problem on neutral atom architectures is challenging, as the field remains at the absolute frontier of quantum computing research. Presently, there is no

universally established protocol or singular "gold standard" architecture. Instead, the literature presents a diverse landscape of experimental approaches and heuristic strategies aimed at overcoming the strict geometric constraints of these platforms.

To illustrate this dynamic ecosystem, recent literature explores a wide array of embedding techniques. For instance, Vercellino et al. [11] investigate the use of Neural Networks to dynamically map problem graphs onto physical registers. Other approaches rely on force-directed heuristics; notably, Tarquini et al. [18] successfully (within a reasonable dimension less than circa 50 nodes) experimented with the Fruchterman-Reingold (FR) algorithm, enhanced by sophisticated pre- and post-processing techniques, to embed Quadratic Unconstrained Binary Optimization (QUBO) problems.

Alongside embedding algorithms, significant research is devoted to translating highly complex real-world use cases into physical layouts. Examples include the formulation of emergency hub placement routing [19] and satellite mission planning [12], demonstrating the vast applicability of QUBO formulations on neutral atoms.

Most important for this thesis, several recent studies have started exploring problem partitioning as a workaround for hardware scale limitations. Works by Nembrini et al. [20,21] and Das et al. [22] propose dividing the original combinatorial problem into sub-instances resolved using localized arrays. While their specific methodologies differ from our scope, these partitioning strategies served as the foundational inspiration for the hybrid *divide et impera* (divide and conquer) architecture proposed in the subsequent chapters of this work.

This diversity of approaches underscores the immaturity and the highly exploratory nature of the field. While the lack of rigid standards offers sig-

nificant opportunities for novel theoretical contributions, it also introduces substantial experimental risks and engineering bottlenecks.

Drawing from this extensive body of literature, we chose to restrict our focus specifically to the QUBO mathematical framework, which will be formally defined in the next section. More importantly, mainly driven by the operational realities and constraints of the currently available experimental platforms, our investigation will target QUBO instances that naturally exhibit a UDG-like connectivity. This specific focus bypasses the need for massive auxiliary embeddings, setting the optimal stage for our distributed algorithmic approach.

2.4 Quadratic Unconstrained Binary Optimization (QUBO)

Quadratic Unconstrained Binary Optimization (QUBO) provides a unifying mathematical framework for modeling a vast array of combinatorial optimization problems. As the name suggests, this formulation strictly involves binary decision variables and a quadratic objective function, notably without imposing any external algebraic constraints.

This formulation has gained immense popularity across various scientific and industrial fields due to its remarkable versatility. As demonstrated in comprehensive tutorials [23], a multitude of classically constrained problems (such as the TSP, Max-Cut, or Graph Coloring) can be seamlessly recast into the unconstrained QUBO format through the use of penalty terms. Consequently, a massive segment of modern literature focuses on QUBO as the standard input format for quantum platforms. While historically popularized by quantum annealing systems built on superconducting qubits (such

as D-Wave [24]), QUBO is increasingly becoming the target formulation for neutral atom architectures as well [11, 12, 22].

From a formal mathematical perspective, solving a QUBO problem entails finding the specific assignment of a binary vector x that minimizes (or maximizes, note that a minimization problem is equivalent to a maximization one by simply applying a minus sign $(-)$ to the objective function) a quadratic cost function:

$$\min_{x \in \{0,1\}^n} f(x) = x^T Q x = \sum_{i=1}^{n} \sum_{j=1}^{n} Q_{i,j} x_i x_j \tag{2.4}$$

where:

- $x = (x_1, x_2, \ldots, x_n)$ is a vector of n binary decision variables;
- Q is an $n \times n$ square matrix of real constants detailing the problem's costs and interactions.

A key mathematical property of binary variables is that $x_i^2 = x_i$ for any $x_i \in \{0, 1\}$. This elegantly allows any linear term of the objective function to be mapped directly onto the main diagonal of the Q matrix ($Q_{i,i}$), while the off-diagonal elements ($Q_{i,j}$ for $i \neq j$) encode the quadratic interactions, or "penalties," between pairs of variables.

Our decision to adopt the QUBO framework for investigating hybrid embedding methodologies is twofold. First, it ensures broad applicability across the wide variety of problems identifiable in the literature [23]. Second, and most crucially, the binary nature of the variables establishes a native, one-to-one mapping with the physical states of a quantum register: each variable $x_i \in \{0, 1\}$ directly dictates whether a specific atom i is left in the ground state ($|0\rangle$) or excited to the Rydberg state ($|1\rangle$). This elegant bridge between

mathematical abstraction and hardware execution perfectly sets the stage for the algorithmic development detailed in the next chapter.

Investigated QUBO Instances Due to specific hardware and simulator constraints, the experimental phase of this work focuses exclusively on UDG-like QUBO matrices. Specifically, we evaluate randomly generated QUBO instances whose underlying interaction connectivity strictly mirrors the geometric topology of a Unit Disk Graph.

Furthermore, to ensure physical compatibility with current neutral atom platforms, these matrices are constrained to exhibit uniform parameter distributions. All elements on the principal diagonal are assigned identical values; mathematically, this uniform linear cost reflects the hardware limitation of employing a global driving laser, which applies the same detuning and Rabi frequency uniformly across the entire atomic register. Similarly, all non-zero off-diagonal elements share a constant penalty value, effectively representing the uniform, binary nature of the Rydberg blockade interaction across all valid UDG edges.

Chapter 3

The Base Pipeline

This chapter presents the baseline architecture of our embedding pipeline and it precedes the related experimental results. The purpose is to detail how the embedding problem was methodologically approached, which embedding optimization methods were selected and tested, and what scalability limitations and fitness scores emerged. Finally, will be discussed how these limitations are natively imposed by the interplay between the mathematical formulation and the specific hardware constraints of the target quantum platform.

3.1 Base pipeline: conceptual scheme

As illustrated in the conceptual scheme in Figure 3.1, the first operational phase of the pipeline requires loading the QUBO problem instance. Therefore, it is strictly necessary to detail the geometric and mathematical characteristics of the investigated instances, as well as the hardware restrictions that dictated their generation.

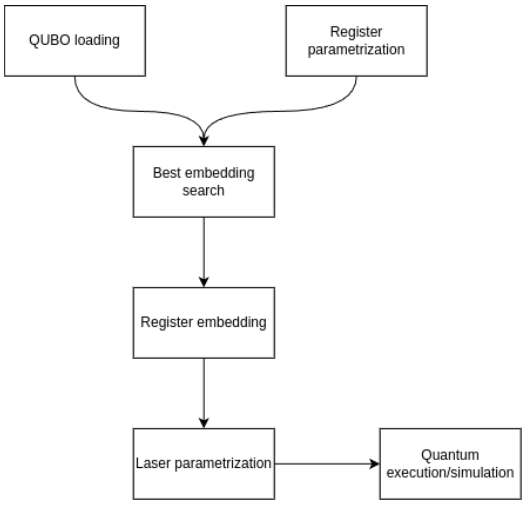

Figure 3.1: Flowchart of the basic pipeline. The process highlights a hybrid classical-quantum approach.

3.1.1 Hardware Constraints and the Jülich HPC Emulator

As discussed, neutral atom platforms present some physical limitations. The most impactful constraint for our formulation is the reliance on a *global driving laser*, which uniformly illuminates the entire atomic register and so imposes the impossibility of applying different excitations to individual atoms or clusters of different atoms. Mathematically, this enables a feasible quantum execution only for QUBO matrices that exhibit uniform values on the principal diagonal (representing a constant linear cost/detuning) and uniform off-diagonal penalties (representing the binary Rydberg blockade).

In order to emulate as much as possible a real-world applicability of this thesis, the pipeline was explicitly designed to target **Jade**, a state-of-the-art Neutral Atom Quantum Processing Unit [25]. While we initially planned for physical execution on it, access to the actual Jade hardware was postponed to the upcoming December due to broader availability constraints.

Consequently, the experimental phase for the moment has relied on a remote analog quantum emulator provided by the **Jülich Supercomputing Centre (JSC)** in Germany.

To ensure that the research performed remains perfectly forward-compatible with the real machine once available, we injected a custom hardware profile into the simulator (referred to in the codebase [26] as `FakeJade`). This profile strictly enforces Jade's actual physical parameters in order to be able to directly test this work on it in the near future.

The `FakeJade` physical profile injected into the emulator severely bounds the degrees of freedom available to the classical embedding algorithm. Extracting directly from platform API call, the hardware constraints are the following:

- **Maximum Radius (R_{max}):** The entire atomic register must be confined within a specific continuous area dictated by the laser focusing limits. For our emulator, the coordinates confined within a circular plane with a maximum radius of $R_{max} = 50.0\,\mu$m.

- **Minimum atomic distance (d_{min}):** Due to the diffraction limits of the optical tweezers, two atoms cannot be placed arbitrarily close or overlapped (we are considering a 2D register). The hardware enforces a minimum inter-atomic distance of $d_{min} = 4.0\,\mu$m. Breaching this threshold causes a hard physical violation and the impossibility to submit the task to remote infrastructure.

- **Interaction Saturation (V_{max}):** Neutral atoms interact following the Van der Waals potential ($V = C_6/r^6$), the combination of physical laws and the d_{min} constraint, caps the maximum possible interaction strength between any two qubits. The highest tolerable penalty in the

system is saturated at $V_{max} = C_6/d_{min}^6$.

- **Global Channel Energetics and Scaling:** The QPU operates via a single `rydberg_global` channel. This imposes ceilings on the maximum allowable Rabi frequency (Ω_{max}) and the maximum absolute detuning ($|\delta|_{max}$). Because raw QUBO weights can mathematically exceed these physical capacities, the embedding pipeline must calculate a restrictive *Scale Factor*. As implemented in the codebase [26], the entire QUBO matrix is dynamically (instance per instance) scaled down by a factor of $\min(\frac{V_{max}}{\max(Q_{off-diagonal})}, \frac{|\delta|_{max}}{\text{avg}(Q_{diagonal})})$ to ensure the problem fits within the hardware's physical energy envelope.

3.1.2 QUBO Instance Generation

These strict hardware parameters played a foundational role in how the test datasets were generated. To guarantee that the problem instances are physically embeddable on the targeted neutral atom architecture, a specialized Python generation script was developed. Rather than generating arbitrary random graphs, the script synthesizes "verisimilar" Unit Disk Graphs (UDG) by simulating the actual physical placement of atoms under the constraints aforementioned. The generation logic is composed of the following procedural steps:

1. **Dynamic Operational Area Calculation:** To achieve a specific target average degree (which dictates primarily the problem's difficulty), an effective operational radius is dynamically calculated. This radius scales proportionally with the square root of the number of nodes (N) and the Rydberg radius. Most important, it is mathematically bounded to never exceed the physical limits of the machine (the $50\,\mu$m maxi-

mum hardware radius) while remaining large enough to accommodate all atoms.

2. **Rejection Sampling Placement:** Node coordinates are generated uniformly within the operational area using polar coordinates. A hardware-aware minimum-distance check is enforced during this process: any randomly generated position that falls within the minimum allowed optical trap distance ($d_{min} = 4.0\,\mu$m) of an already placed atom is immediately rejected. This ensures the resulting spatial configuration respects the optical tweezers' constraints.

3. **QUBO Matrix Construction:** Once all N coordinates are successfully placed within the delimited continuous space, the pairwise Euclidean distance matrix is computed. The QUBO matrix (Q) is then populated: all main diagonal elements are uniformly set to -1.0 (representing the global laser's tendency to drive the atoms toward the Rydberg state), and a constant penalty weight of 2.0 is assigned to any off-diagonal edge where the inter-atomic distance is strictly less than the specified Rydberg blockade radius set ($R_b = 12.0\,\mu$m).

4. **Data Serialization:** Finally, the non-zero upper triangular elements of the QUBO matrix are extracted and serialized into a (`.npz`) format. Although the original geometrically valid coordinates are stored for validation purposes, the embedding pipeline receives only the abstract mathematical weights (otherwise all of the next experimentations would not make much sense), forcing the selected embedding algorithm to reconstruct a valid physical layout from scratch.

```
=== MATRIX VALUES INSPECTION: jade_udg_10x10_R12_s52_d5.0.npz ===
Size: 10x10

          0     1     2     3     4     5     6     7     8     9
------------------------------------------------------------------
 0 |  -1.00  0.00  2.00  0.00  0.00  0.00  0.00  0.00  2.00  0.00
 1 |   0.00 -1.00  2.00  2.00  2.00  0.00  2.00  0.00  0.00  2.00
 2 |   2.00  2.00 -1.00  0.00  0.00  0.00  0.00  2.00  2.00  0.00
 3 |   0.00  2.00  0.00 -1.00  0.00  2.00  0.00  0.00  0.00  2.00
 4 |   0.00  2.00  0.00  0.00 -1.00  0.00  2.00  2.00  0.00  0.00
 5 |   0.00  0.00  0.00  2.00  0.00 -1.00  0.00  0.00  0.00  2.00
 6 |   0.00  2.00  0.00  0.00  2.00  0.00 -1.00  0.00  0.00  0.00
 7 |   0.00  0.00  2.00  0.00  2.00  0.00  0.00 -1.00  0.00  0.00
 8 |   2.00  0.00  2.00  0.00  0.00  0.00  0.00  0.00 -1.00  0.00
 9 |   0.00  2.00  0.00  2.00  0.00  2.00  0.00  0.00  0.00 -1.00

--- Embedding Stats ---
Graph density:           31.1% (14/45 edges)
Nodes degree (Min/Max):  2 / 5 (Average: 2.8)
Quadratic Weights Range: From 2.0000 to 2.0000
Dynamic Range (Max/Min): 1.0x
Std Deviation (Edges):   0.0000
Diagonal Mean (Lin):     1.0000
```

Figure 3.2: Example of a small-scale UDG-like QUBO instance generated using the *instance_generator.py* as described above and inspected via *inspect_qubo.py* in [26].

3.1.3 Instance Loading and Matrix Parsing

Once generated, the QUBO matrices and their metadata are compressed and stored in NumPy's binary format (`.npz`). As depicted in the first block of the pipeline flowchart, the execution begins with the `QUBO loading` module.

A dedicated parser function extracts the indices and weights from the `.npz` file and reconstructs the symmetric $N \times N$ matrix (Q). Simultaneously, the system isolates the off-diagonal terms to calculate the maximum theoretical connectivity.

3.2 Embedding Optimization using Genetic Algorithms (GA)

The search for an optimal physical embedding is a NP-Hard task characterized by a highly complex, non-convex optimization scenario that makes the use of classical optimization methods, such as the Nelder-Mead simplex method, highly ineffective. As formalized in the previous section, among

the challenges, we have that the computational engine must simultaneously satisfy strict geometric boundaries (R_{max}) and discrete proximity constraints (d_{min}) for example, all while minimizing in some way the topological error between the physical Van der Waals interactions and the mathematical QUBO penalties in order to achieve a good enough fitness score and consequently (at least in principle) a good enough embedding.

Given the severity of these constraints and the complete lack of *a priori* analytical information regarding the optimal spatial distribution of the atoms, we chose to initiate our experimental phase by testing a black-box, derivative-free optimization approach: Genetic Algorithms (GAs). The main goal of this section is to explain the core characteristics of GAs, in order to better contextualize our experimental results.

3.2.1 Genetic Algorithm Framework

Genetic Algorithms represent a prominent branch of evolutionary computation, heavily inspired by the biological principles of natural selection and genetics. While methodological classifications and terminologies vary significantly depending on the specific biological mechanisms inspiring them, GAs are universally recognized as highly effective for optimization tasks characterized by discontinuous search spaces and a complete lack of *a priori* analytical information.

In our specific context of application, and following a comprehensive study of the foundational literature [27], we designed our GA architecture around the following key structural principles:

- **Chromosome Encoding:** Unlike classical binary GAs, an individual (or chromosome) in our population is encoded as a continuous, real-

valued array of 2D spatial coordinates (x, y). This array strictly defines the physical placement of all N atoms within the 2D register.

- **Evolutionary Operators:** The population is iteratively evolved across generations utilizing customized selection, crossover, and mutation mechanisms.

- **Evaluation Metric:** The survival and reproduction probability of each layout is strictly governed by a custom Fitness Function, which evaluates the topological validity and quality of the configuration.

3.2.2 GA Parametrization: Explanation and Example

In this subsection, we present a baseline parametrization example of our Genetic Algorithm. The primary purpose is to detail the specific hyperparameter configuration and the evolutionary mechanisms leveraged. This specific setup represents an example of our algorithmic foundation in our approach, because also with larger or different instances the core architecture remains identical, requiring only dynamic (and/or manual) adjustments to the parameters (for example, applying more aggressive mutation rates or larger population sizes for higher-dimensional topologies). For illustrative purposes, below we report the configuration for a 5x5 instance (N=5, 5 nodes).

Table 3.1 illustrates the exact configuration adopted in the *bench-GA.py* script from the project codebase [26]. This setup gives us the opportunity to explicitly justify our design choices, which were heavily guided by the theoretical foundations established in [27].

To effectively explore the continuous geometric search space (which is inherently vast and intractable), the evolutionary pressure is driven by a

GA Parameter	**Chosen Value**	**Conceptual Meaning**
Chromosome Length	$2N$	Continuous 2D coordinates for all N atoms.
Population Size	100	Number of candidate layouts per generation.
Max Generations	800	Termination criterion for the evolutionary loop.
Fitness Function	Custom	Evaluates spatial topology against the target QUBO matrix.
Selection Strategy	Tournament ($K = 3$)	Selects parents ensuring high competitive pressure.
Mating Pool Size	20	Number of parents selected for reproduction.
Crossover Operator	Uniform	Exchanges individual (x, y) coordinates between parents.
Mutation Strategy	Adaptive	Dynamically scales mutation probability (40% to 5%) based on fitness.
Mutation Bounds	$[-3.0, 3.0]$ μm	Spatial perturbation window for local coordinate fine-tuning.
Elitism	5	Preserves the 5 best layouts across generations without alterations.

Table 3.1: Hyperparameter baseline configuration for the Genetic Algorithm setup for a 5x5 instance (N=5 nodes).

Tournament Selection mechanism. This method selects the parent individuals for the mating phase while maintaining sufficient genetic diversity, thereby avoiding premature convergence. At the same time, to preserve high-quality geometric structures discovered during the search, an elitism strategy was adopted. This ensures that the best-performing embeddings are passed completely unaltered to the subsequent generation; otherwise, spatial mutations could easily disrupt these high-quality candidates.

Regarding the genetic operators, a Uniform Crossover was implemented to exchange individual coordinates between parents, since traditional crossover methods based on splitting continuous arrays would disrupt the geometric integrity of the spatial layouts. However, the most critical configuration involved the mutation operator. Given the continuous nature of our coordinates and the strict precision required by the d_{min} physical constraint, an Adaptive Mutation strategy was deployed. As detailed in the table, the mutation probability dynamically scales between 40% and 5% based on the individual's fitness, applying a spatial perturbation within a strict $[-3.0, 3.0]$ μm window. This adaptive approach is fundamentally advantageous: it allows the algorithm to aggressively explore the space during early generations and perform a delicate local fine-tuning as the population converges toward valid physical solutions. In this way, we can rapidly evaluate the enormous initial set of candidates in the solution landscape, increasing the probability of finding viable starting points.

It is important to emphasize once again that designing and tuning a Genetic Algorithm must adhere to theoretical guidelines, but it also relies heavily on empirical craftsmanship and domain expertise. Understanding the specific physical constraints of the application problem is crucial for adapting the GA effectively. Consequently, while we cannot claim this to be the

absolute optimal configuration mathematically, it certainly represents the most effective setup achieved during our experimental phase.

This empirical nature is evident in our choices: instead of terminating based on a target fitness score or execution time, we opted for a fixed maximum number of generations (stopping criterion) to better accommodate the stochastic nature of GAs. Furthermore, because every evolutionary run is inherently probabilistic, the algorithm is executed multiple times (multi-shot) for each instance to ensure robust and statistically significant results. The best-score run is selected.

Finally, these observations highlight one of the primary drawbacks of Genetic Algorithms: identifying a robust parametrization requires significant time dedicated to hyperparameter fine-tuning [27]. It demands extensive experimentation to develop a heuristic sensitivity to how the algorithm responds to varying problem sizes, graph topologies, and interaction densities.

3.2.3 Fitness Function Formulation

The core of the evolutionary optimization is driven by the evaluation metric, represented in our algorithm by the *Improved Topological Mean Absolute Error (MAE)* fitness function. This custom function calculates the geometric "quality" of a candidate layout and its structural isomorphism to the target QUBO matrix, heavily penalizing configurations that violate the hardware's physical constraints.

While fitness functions are inherently flexible from an algorithmic design perspective and can be personalized to obtain a metric score tailored for specific combinatorial problems, our implementation (documented in the *bench-GA.py* script [26]) adopts a strict physics-aware approach. Although multiple fitness functions and their variants were implemented and extensively tested

during the experimental phase, the *Improved Topological MAE* consistently yielded the highest quality embeddings. The mathematical design of this optimally selected metric is formalized by the following foundational parameters:

- Let N be the number of logical variables (atoms).
- An individual in the population is defined by a set of spatial coordinates $\mathbf{r}_i \in \mathbb{R}^2$ for $i = 1, \ldots, N$.
- The Euclidean distance between any pair of atoms is denoted as $r_{ij} = ||\mathbf{r}_i - \mathbf{r}_j||$ with $i \neq j$.
- The physical interaction potential between two atoms is strictly governed by the Van der Waals force, expressed as $V_{ij} = C_6/r_{ij}^6$.

A critical computational challenge in continuous spatial embeddings is the numerical instability triggered when two atoms are placed too close to each other. As the distance $r_{ij} \to 0$, the physical potential V_{ij} explodes to infinity, effectively destroying the evaluation for the entire array. To circumvent this, the maximum physical potential must be analytically clipped, defining the maximum desired interaction from the scaled target QUBO matrix Q as $Q_{max} = \max_{i<j}(|Q_{ij}|)$, the effective physical potential $\tilde{V}_{ij}$ is bounded as:

$$\tilde{V}_{ij} = \min\left(\frac{C_6}{r_{ij}^6},\ n \cdot Q_{max}\right)$$

with $n = 5$ acting as a saturation threshold in our configuration.

To accurately evaluate the topological mismatch between the physical layout and the mathematical abstraction, we partition the upper-triangular interactions into two disjoint sets. The active bounds $E_{active} = \{(i, j) \mid |Q_{ij}| > \epsilon\}$ represent the necessary logical connections, while the inactive

bounds $E_{inactive} = \{(i,j) \mid |Q_{ij}| \leq \epsilon\}$ represent pairs that must not physically interact. A tolerance of $\epsilon = 10^{-5}$ is introduced to safely handle floating-point inaccuracies. This explicit classification ensures that the algorithm can accurately shape the required couplings while actively filtering out the noise.

Consequently, the topological base error (E_{base}) is computed by measuring the exact deviation on the active bounds, while simultaneously applying a heavy penalty for any unwanted crosstalk on the inactive ones:

$$E_{base} = \sum_{(i,j)\in E_{active}} \left|\tilde{V}_{ij} - Q_{ij}\right| + \lambda \sum_{(i,j)\in E_{inactive}} \tilde{V}_{ij}$$

where $\lambda = 2.0$ is an amplification multiplier that forces the evolutionary algorithm to prioritize moving unconnected nodes apart.

Beyond the topological layout, the configuration must strictly adhere to the Neutral Atom hardware specifications. Specifically, we must guarantee a minimum inter-atomic distance ($d_{min} = 4.0$ μm) to respect the optical tweezers' diffraction limits, and confine the entire register within a maximum radius ($R_{max} = 50.0$ μm) dictated by the global laser's boundaries. These hard physical constraints are enforced through a severe linear penalty function P:

$$P = K \left(\sum_{i<j} \max(0, d_{min} - r_{ij}) + \sum_{i} \max(0, ||\mathbf{r}_i|| - R_{max}) \right)$$

where $K = 10^5$ acts as a massive penalty factor that immediately drops physically unfeasible candidates out of the evolutionary mating pool.

Since Genetic Algorithms are natively designed to maximize a specific score, the accumulated topological error and physical penalties are inverted to yield a strictly positive final fitness value F:

$$F = \frac{1}{E_{base} + P + \delta}$$

The minimal constant $\delta = 10^{-6}$ is introduced simply to prevent zero-division crashes in the extremely unlikely event that the algorithm discovers a mathematically perfect continuous mapping.

It is straightforward also from above how there is not a single valid method or theory to build "the best" fitness function.

3.3 Laser Parametrization

The laser parametrization phase could easily warrant an entire thesis in itself. The current literature presents a multitude of proposed variants and quantum algorithms with diverse parametrization strategies, and a universally accepted gold standard has not yet been established. For instance, while many studies rely on a foundational adiabatic approach, numerous advanced variants are actively being explored, as demonstrated in [28].

In our case, since the core focus of this thesis is centered on the geometric embedding optimization and the hybrid architecture, we adopted a robust but streamlined approach to the quantum processing phase. We designed the laser parametrization to a level of depth that guarantees physical validity and efficient analog execution, without overcomplicating the pulse sequences as suggested and explained in [29]. Consequently, we fixed the following driving parameters:

- **Pulse Duration (T):** The total execution time of the analog quantum sequence is strictly fixed at $T = 4000$ ns ($4.0\,\mu$s). This temporal window provides a sufficient timeframe for the adiabatic evolution while remaining well within the coherence time limits of the hardware. This parametrization decision is based on the prescription of Adiabatic Theorem, which dictates a slow and gradual evolution avoiding transitions

to unwanted excited states [8]. An alternative can be the Quantum Approximate Optimization Algorithm (QAOA) [29], which uses ultra-short, discretized laser pulses.

- **Rabi Frequency Profile (Ω):** To induce the quantum transitions, the amplitude pulse is shaped as a smooth interpolation that starts at 0, peaks at a specific maximum value Ω, and returns to 0. To ensure the stability of the global laser, Ω is dynamically calculated as the median of the positive QUBO target weights, but it is strictly capped at 83.3% of the hardware's absolute maximum amplitude capability. This conservative engineering choice prevents hardware saturation, whereas more aggressive alternatives would require complex Optimal Control Theory (OCT) pulse-shaping algorithms.

- **Detuning Sweep (δ):** The core of the adiabatic execution is enforced via a linear detuning sweep. The frequency profile starts at a negative initial value $\delta_0 = -\Delta$ (forcing the system into a trivial ground state), linearly crosses the resonance frequency, and ends at a symmetrical positive value $\delta_f = +\Delta$ to drive the atoms towards the Rydberg state [9]. The magnitude of Δ is dynamically scaled instance by instance. While non-linear sweeps could theoretically improve the ground-state probability by slowing down near the minimum energy gap, the linear sweep provides the most robust and standard baseline for evaluating our geometric embeddings.

This standardized adiabatic protocol guarantees that the quantum many-body system is initialized in an easy-to-prepare ground state and is smoothly driven toward the complex energy landscape encoded by our custom physical layout. We started from [29] to study how to handle QUBO problems

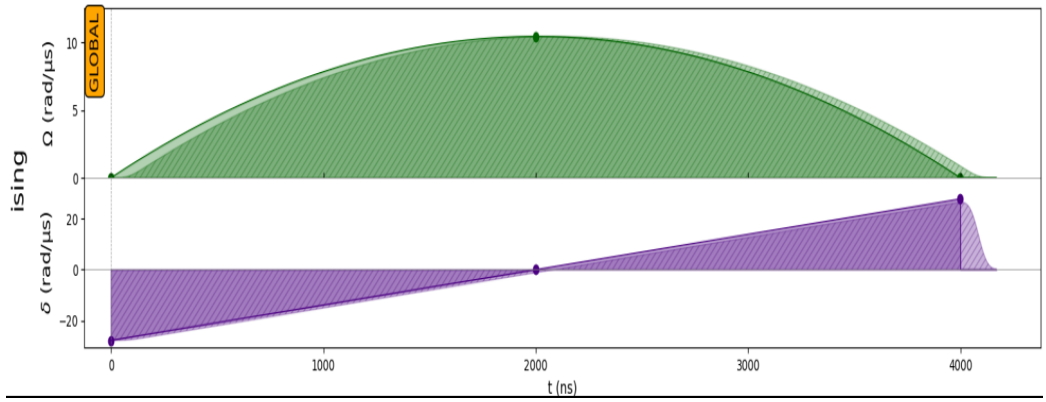

Figure 3.3: Visual representation of the analog adiabatic pulse sequence generated via the Pulser framework. The top panel (green) illustrates the smooth interpolated profile of the Rabi frequency Ω, strictly capped below the hardware threshold. The bottom panel (purple) displays the symmetric linear sweep of the detuning δ across the resonance frequency, executed over the total $T = 4.0\,\mu$s temporal window. The 'GLOBAL' label indicates the use of a single global laser beam acting on the entire atomic register.

on this platform, but the entire Pasqal documentation ecosystem is highly comprehensive and can be useful to read. For instance, an alternative tutorial dedicated to Maximum Weight Independent Set (MWIS) problems [30] provides further insights on different problmes' resolution.

Chapter 4

GAs: Experimental Results and Limitations

While numerous fitness functions, hyperparameters, and QUBO instances were experimented with and validated during the research phase, this section reports the most prominent results to highlight the critical operational aspects in using Genetic Algorithm with our embedding architecture.

4.1 Validation on a 5x5 Instance

To rigorously validate the embedding model, the experimental phase was initiated with a relatively small-scale 5×5 instance, as depicted in Figure 4.1.

This specific configuration serves as an ideal baseline candidate. Despite its small dimensionality, it features a non-trivial density and a mathematically interesting topology composed of disconnected sub-graphs: a two-node island connecting q_0 and q_2, and a separate localized chain linking q_1, q_3, and q_4.

```
=== MATRIX VALUES INSPECTION: jade_udg_5x5_R12_s47.npz ===
Size: 5x5

          0     1     2     3     4
-----------------------------------
 0 |  -1.00  0.00  2.00  0.00  0.00
 1 |   0.00 -1.00  0.00  2.00  2.00
 2 |   2.00  0.00 -1.00  0.00  0.00
 3 |   0.00  2.00  0.00 -1.00  0.00
 4 |   0.00  2.00  0.00  0.00 -1.00

--- Embedding Stats ---
Graph density:           30.0% (3/10 edges)
Nodes degree (Min/Max):  1 / 2 (Average: 1.2)
Quadratic Weights Range: From 2.0000 to 2.0000
Dynamic Range (Max/Min): 1.0x
Std Deviation (Edges):   0.0000
Diagonal Mean (Lin):     1.0000
```

Figure 4.1: Structure of the small-scale UDG-like QUBO instance ($N = 5$) generated via *instance_ generator.py*, inspected using *inspect_ qubo.py*, and subsequently embedded using the *bench-GA.py* module [26].

The Genetic Algorithm was executed utilizing a multi-start approach consisting of 5 independent runs. This specific volume of restarts was empirically established as an optimal trade-off between maximizing the probability of finding the global spatial optimum and maintaining a manageable classical execution time, thereby mitigating the inherent stochastic nature of evolutionary algorithms. The hyperparameters employed for this specific dimensional scale are detailed in Table 4.1.

As evident from the table, the parametrization is not excessively aggressive. Because the spatial search space for a 5-node matrix is relatively contained, extremely high mutation parameters are not required to avoid local minima. Under this configuration, the best evolutionary run achieved an excellent spatial fitness score of 0.3968, demanding a total classical CPU execution time of 144.35 seconds. Most importantly, this run generated the geometrically feasible embedding illustrated in Figure 4.2.

The physical layout intuitively respects the mathematical constraints previously analyzed, properly isolating the distinct logical islands while strictly adhering to the minimum inter-atomic distance and maximum global radius

PyGAD Parameter	**Configured Value**	**Conceptual Meaning**
`num_genes`	$2 \times N_{atoms}$	Chromosome length: represents continuous 2D coordinates (x, y) for all atoms.
`sol_per_pop`	100	Population size: number of candidate physical layouts per generation.
`num_generations`	800	Termination criterion for the evolutionary loop.
`fitness_func`	`improved_topological_MAE`	Custom evaluation metric (incorporates topological error and hardware physical penalties).
`parent_selection_type`	Tournament	Selection strategy to maintain competitive evolutionary pressure.
`K_tournament`	3	Number of candidate individuals competing in each selection tournament.
`num_parents_mating`	20	Number of parents successfully selected to compose the mating pool.
`crossover_type`	Uniform	Exchanges individual (x, y) coordinates between parents without disrupting array bounds.
`mutation_type`	Adaptive	Dynamically scales mutation rate to balance exploration and fine-tuning.
`mutation_probability`	$[0.4, 0.05]$	Assigns a 40% mutation rate to low-fitness individuals and 5% to elite ones.
`random_mutation_min/max`	$[-3.0, 3.0]$	Continuous spatial perturbation window (in μm) for local coordinate adjustments.
`keep_elitism`	5	Preserves the absolute best 5 layouts across generations without any genetic alteration.
`allow_duplicate_genes`	False	Prevents multiple atoms from collapsing into the exact same mathematical coordinates.

Table 4.1: Exact hyperparameter configuration adopted for the Genetic Algorithm setup using the PyGAD library, referencing the base architecture for a 5×5 problem embedding.

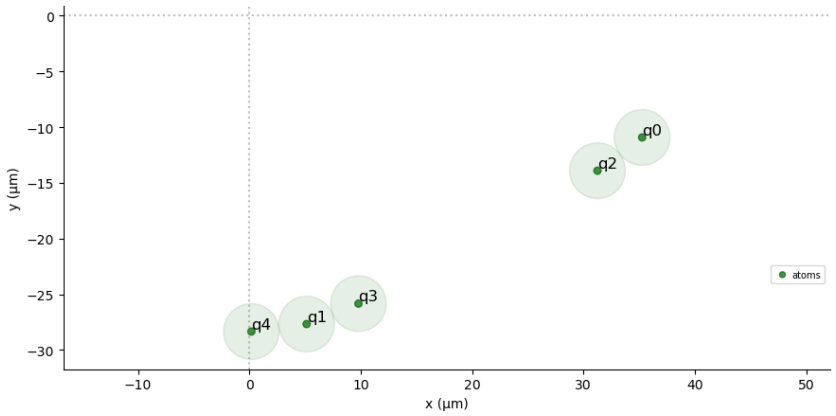

Figure 4.2: The 2D physical spatial layout generated by the Genetic Algorithm for the $N = 5$ instance. The register fully complies with Neutral Atom QPU geometric boundaries.

imposed by the hardware profile. When these physical coordinates were compiled into an adiabatic sequence and submitted to the QPU emulator, the system produced the output distribution shown in Figure 4.3.

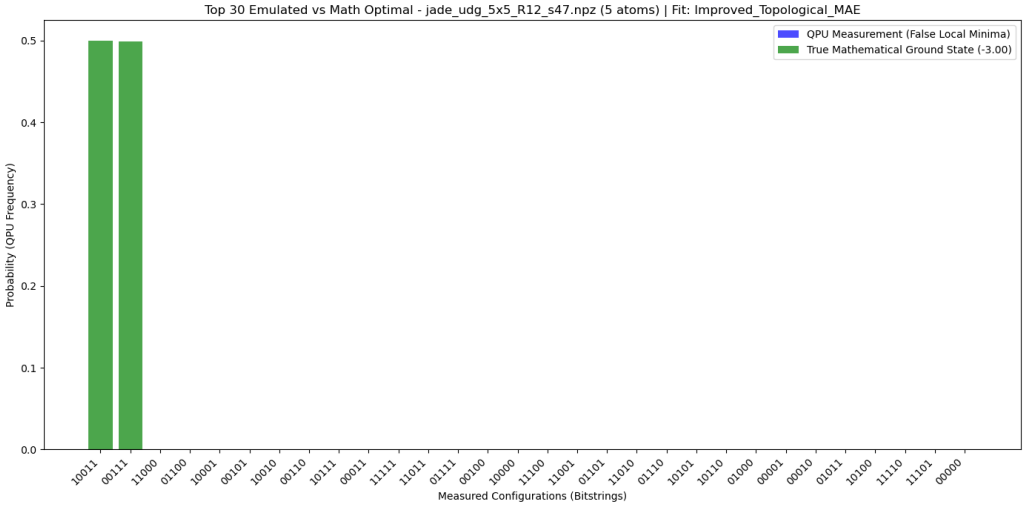

Figure 4.3: Probability distribution of the measured bitstrings after the adiabatic execution on the QPU emulator. The highest probability peaks correspond perfectly to the optimal solutions of the original QUBO instance.

The quantum execution demonstrates outstanding performance: the ana-

log platform identified the two optimal ground states found with classical brute-force method (suitable for small instances like in this case) with exceptionally high probability and symmetry. This initial validation serves as a highly encouraging proof-of-concept for our hybrid pipeline, setting the baseline to systematically investigate the scalability limits of this exact mathematical approach.

4.2 Scaling Validation on a 6x6 Instance

Continuing the systematic scaling tests, the problem dimensionality was incrementally increased. For the 6×6 dimensional scale, the target matrix is depicted in Figure 4.4.

```
=== MATRIX VALUES INSPECTION: jade_udg_6x6_R12_s48.npz ===
Size: 6x6

          0     1     2     3     4     5
----------------------------------------
 0 |  -1.00  2.00  0.00  0.00  2.00  2.00
 1 |   2.00 -1.00  2.00  0.00  2.00  0.00
 2 |   0.00  2.00 -1.00  2.00  0.00  0.00
 3 |   0.00  0.00  2.00 -1.00  0.00  0.00
 4 |   2.00  2.00  0.00  0.00 -1.00  0.00
 5 |   2.00  0.00  0.00  0.00  0.00 -1.00

--- Embedding Stats ---
Graph density:           40.0% (6/15 edges)
Nodes degree (Min/Max):  1 / 3 (Average: 2.0)
Quadratic Weights Range: From 2.0000 to 2.0000
Dynamic Range (Max/Min): 1.0x
Std Deviation (Edges):   0.0000
Diagonal Mean (Lin):     1.0000
```

Figure 4.4: Structure of the 6×6 UDG-like QUBO instance generated via *instance_ generator.py*, inspected using *inspect_ qubo.py*, and subsequently embedded using the *bench-GA.py* module [26].

This specific configuration presents an increased graph density and a maximum node degree of 3, introducing topological constraints that are significantly less trivial than those encountered in the 5×5 instance. To properly explore the vastly expanded continuous search space and prevent premature convergence, the Genetic Algorithm's hyperparameters required a more ag-

gressive tuning, as detailed in Table 4.2.

PyGAD Parameter	Updated Value	Conceptual Meaning
`sol_per_pop`	200	Population size doubled (from 100) to enhance genetic diversity and broaden the exploration capabilities within a vaster search space.
`num_generations`	1500	Increased termination limit (from 800) to grant the evolutionary engine sufficient time to converge on harder topologies.
`fitness_func`	`improved_topological_MAE`	Transitioned to the refined metric (handling potential clipping and inactive bounds penalties) to better guide complex layouts.

Table 4.2: Boosted hyperparameter configuration for more complex QUBO instances. All other evolutionary operators and parameters not explicitly listed here remain strictly identical to the baseline configuration previously defined in Table 4.1.

To further compensate for the structural difficulty of the matrix, the number of independent algorithm restarts was increased to 10. Under these constrained conditions, the best spatial embedding achieved is shown in Figure 4.5.

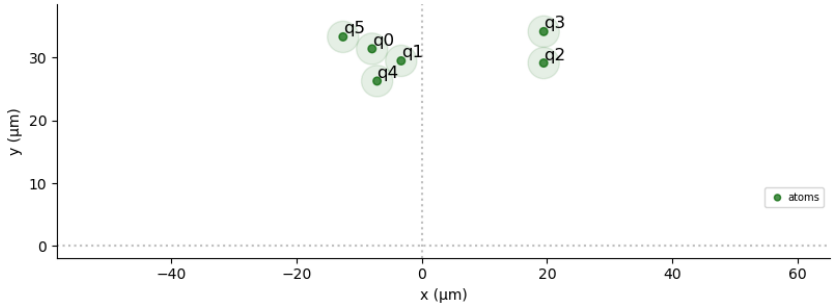

Figure 4.5: The 2D physical spatial layout generated by the Genetic Algorithm for the $N = 6$ instance. The register fully complies with Neutral Atom QPU geometric boundaries.

This configuration yielded a relatively low best spatial fitness score of 0.0136, demanding a classical computational execution time of 1176.26 sec-

onds. When submitted to the analog emulator, the quantum execution produced the measurement distribution reported in Figure 4.6.

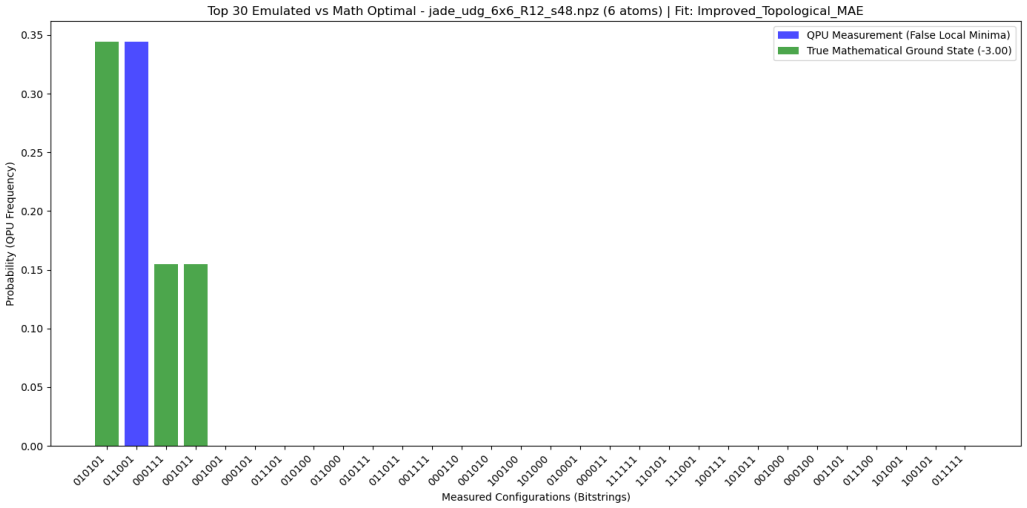

Figure 4.6: Probability distribution of the measured bitstrings after the adiabatic execution. Unlike the smaller instance, the highest probability peak corresponds to a false local minimum, while the true mathematical optimal solutions retain a lower, yet sufficient, cumulative probability mass.

Analyzing the probability distribution, a critical phenomenon emerges: the highest probability peak no longer corresponds to the true mathematical ground state, but rather to an excited state (a false local minimum). However, the true optimal solutions are still identified with a sufficiently high cumulative probability mass. Since quantum computation ultimately relies on statistical sampling, successfully isolating the optimal energy solution among the measured bitstrings constitutes a valid problem resolution.

Crucially, this result highlights the *analog resilience* of the Neutral Atom QPU. It demonstrates that finding a mathematically "perfect" embedding is not strictly necessary; a "good enough" layout (even with a low fitness score of 0.0136) can still guide the quantum dynamics toward the global optimum with an acceptable degree of success.

While the classical execution time required to compute this embedding ($\sim$ 20 minutes) is substantial, it must be contextualized. Firstly, it heavily depends on the consumer-grade hardware utilized for the preprocessing phase (refer to Appendix A for local hardware specifications). Secondly, at this preliminary stage of the research, the primary objective is validating the physical capability of the architecture rather than optimizing its classical runtime overhead.

4.3 Scaling Analysis: A 7x7 Instance

This specific instance is presented to highlight an interesting consideration regarding problem difficulty and embedding complexity. The mathematical formulation of the target problem is shown in Figure 4.7.

```
=== MATRIX VALUES INSPECTION: jade_udg_7x7_R12_s49.npz ===
Size: 7x7

          0     1     2     3     4     5     6
-----------------------------------------------
 0 |  -1.00  0.00  0.00  2.00  0.00  0.00  0.00
 1 |   0.00 -1.00  0.00  0.00  0.00  2.00  0.00
 2 |   0.00  0.00 -1.00  0.00  2.00  0.00  0.00
 3 |   2.00  0.00  0.00 -1.00  0.00  0.00  0.00
 4 |   0.00  0.00  2.00  0.00 -1.00  0.00  0.00
 5 |   0.00  2.00  0.00  0.00  0.00 -1.00  0.00
 6 |   0.00  0.00  0.00  0.00  0.00  0.00 -1.00

--- Embedding Stats ---
Graph density:           14.3% (3/21 edges)
Nodes degree (Min/Max):  0 / 1 (Average: 0.9)
Quadratic Weights Range: From 2.0000 to 2.0000
Dynamic Range (Max/Min): 1.0x
Std Deviation (Edges):   0.0000
Diagonal Mean (Lin):     1.0000
```

Figure 4.7: Structure of the 7×7 UDG-like QUBO instance generated via *instance_ generator.py*, inspected using *inspect_ qubo.py*, and subsequently embedded using the *bench-GA.py* module [26].

It is important to note that while a larger number of nodes typically implies a higher spatial embedding difficulty, this specific matrix exhibits a lower graph density and a reduced maximum node degree compared to the

previous 6×6 instance. As will be demonstrated, these specific topological traits significantly facilitate the algorithmic layout process.

Consequently, despite the increased dimensionality, the Genetic Algorithm was executed using a less aggressive configuration, altering only a single parameter compared to the baseline 5×5 setup, as detailed in Table 4.3.

PyGAD Parameter	**Updated Value**	**Conceptual Meaning**
`fitness_func`	`improved_topological_MAE`	Retained the refined metric (handling complex bounds and hardware penalties) to guide the layout effectively.

Table 4.3: Hyperparameter configuration for further scaling tests. To isolate the impact of the evaluation metric and optimize classical execution times, the population size and generation limits were reverted to their original baseline values (100 and 800, respectively, as in Table 4.1.

Executing the algorithm under these parameters yielded the spatial embedding illustrated in Figure 4.8.

The physical layout reveals a highly structured and well-spaced geometric arrangement. Remarkably, the classical execution time was only 180.92 seconds, achieving a very strong spatial fitness score of 2.266. This excellent mathematical convergence set the expectation for an optimal quantum solution retrieval, which was subsequently confirmed by the QPU execution shown in Figure 4.9.

The quantum execution demonstrates perfectly symmetrical and outstanding results, effortlessly isolating the global optima. This instance is reported specifically to prove that the overall embedding difficulty is not dictated solely by the matrix dimensionality (N), but rather by a complex interplay of graph density, node degree distribution, and structural topology. Indeed, we observed that for this larger 7×7 grid, the hybrid pipeline ob-

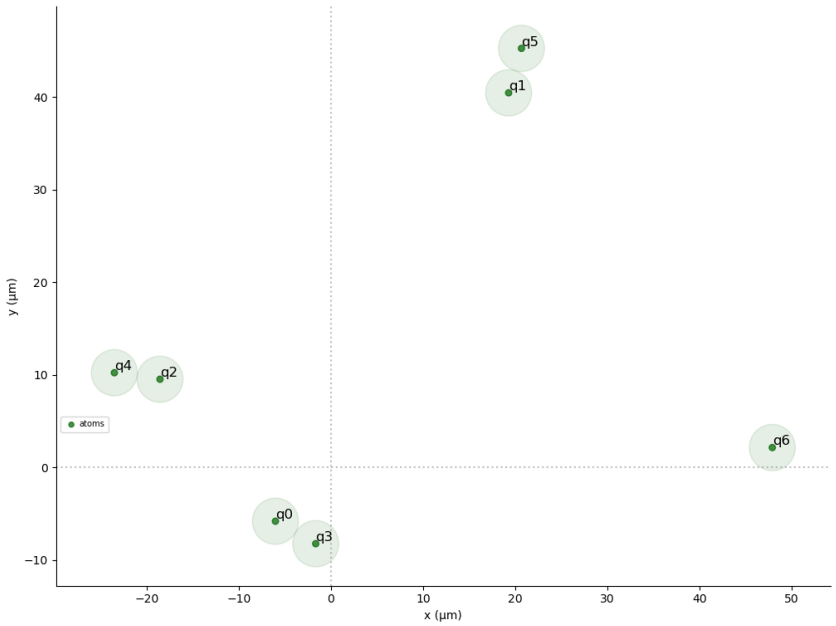

Figure 4.8: The 2D physical spatial layout generated by the Genetic Algorithm for the $N = 7$ instance. The register fully complies with Neutral Atom QPU geometric boundaries.

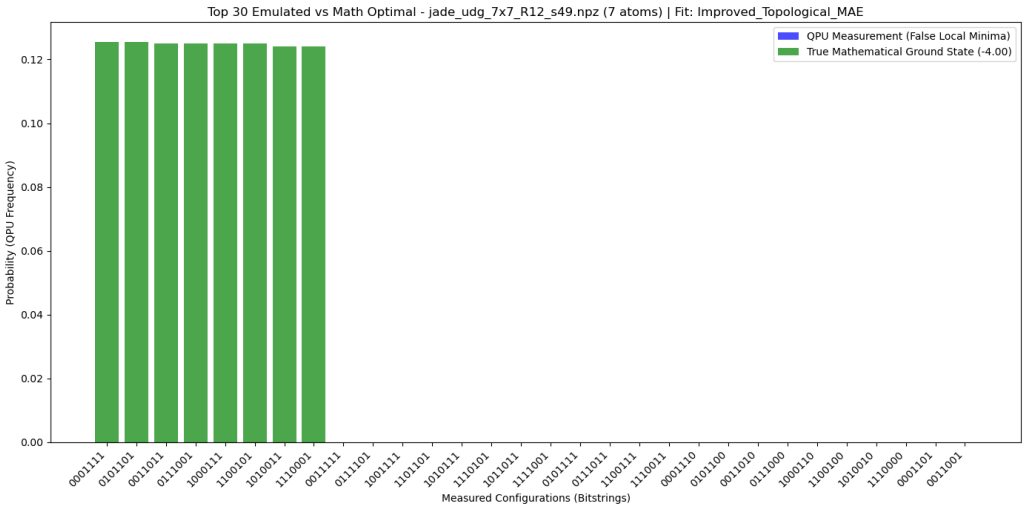

Figure 4.9: Probability distribution of the measured bitstrings after the adiabatic execution. The analog system successfully identifies the true mathematical ground states with a dominant probability mass, indicating a highly effective spatial embedding.

tained significantly better classical preprocessing times and quantum readout results than it did for the smaller, yet denser, 6×6 problem.

4.4 10x10 instance

4.5 15x15 instance

We have surely to consider that all the classical part has been executed on my laptop with the hardware specification in Appendix A A, with a better hardware and possibly a HP classical hardware capacity we would have been able, within the same execution time, to embed bigger matrices, but it is however clear how in a "common" configuration of execution Genetic Algorithm imposes severe limitations in scalability. For this reasons other embedding methods has been studied and experimented and for the good results shown, Simulated Annealing will be presented in the following.

Appendix A

Classical Hardware

To ensure the scientific reproducibility of the classical executions presented throughout this thesis, specifically concerning the preprocessing execution times of the Genetic Algorithm and the Simulated Annealing optimization, the precise hardware and software specifications of the local machine utilized for all classical computational tasks are detailed in Table A.1.

Component / Environment	**Specification**
Processor (CPU)	AMD Ryzen 7 3700U (4 Cores, 8 Threads)
CPU Clocks	Base: 2.3 GHz, Max Boost: 4.0 GHz
CPU Cache	L1: 96 KB/core, L2: 512 KB/core, L3: 4 MB shared
Memory (RAM)	20 GB DDR4 @ 2400 MHz (4 GB SODIMM + 16 GB)
Operating System	Ubuntu 22.04 LTS

Table A.1: Hardware and OS configuration of the local machine used for the classical embedding optimization and preprocessing phases.

This appendix principally has the scope to contextualize the execution time observed, for this reason is important to note that these specifications refer exclusively to the classical operations (embedding optimization and sequence compilation). The actual quantum executions were asynchronously submitted to the remote analog quantum emulator provided by the Jülich Supercomputing Centre, which relies on a dedicated HPC architecture.

Appendix B

AI Usage Policy and Declaration

The purpose of this appendix is to declare, in the interest of academic integrity and in strict accordance with current university directives, how Large Language Models (LLMs) and Artificial Intelligence (AI) were utilized during the experimental research and drafting of this thesis.

Specifically, a Gemini Advanced/Pro tier account, accessible for educational purposes, was employed. AI technologies were never utilized as a substitute for original critical thinking, nor were they employed in any improper or unauthorized manner. Throughout the entire workflow, the AI served exclusively as an auxiliary tool to brainstorm concepts, suggest structural refinements, and assist in generating or debugging code snippets (which were subsequently subjected to rigorous manual review and validation).

Every interaction with the AI was conducted under the direct input, constant supervision, and final editorial judgment of the candidate, ensuring full alignment with the methodologies prescribed by the academic institution and the thesis supervisors. Ultimately, integrating these technologies into the research pipeline not only streamlined the technical execution but also provided a valuable opportunity to evaluate the current capabilities and

limitations of modern AI tools in an advanced academic context.

Bibliography

[1] Q. A. Memon, M. A. Ahmad, and M. Pecht, "Quantum computing: Navigating the future of computation, challenges, and technological breakthroughs," *Quantum Reports*, vol. 6, no. 4, pp. 627–663, 2024.

[2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary ed., 2010.

[3] Q. A. Memon, M. Al Ahmad, and M. Pecht, "Quantum computing: Navigating the future of computation, challenges, and technological breakthroughs," *Quantum Reports*, vol. 6, pp. 627–663, 2024.

[4] M. Grotti, S. Marzella, G. Bettonte, D. Ottaviani, and E. Ercolessi, "Practical use cases of neutral atoms quantum computers," 2026.

[5] Google, "What is quantum computing?," whitepaper, Google, 2024. Guide for policymakers.

[6] Microsoft, "Microsoft's majorana 1 chip carves new path for quantum computing," 2024.

[7] C. C. McGeoch and P. Farré, "The d-wave advantage system: An overview," Technical Report 14-1049A-A, D-Wave Systems Inc., 2020.

[8] L. Henriet, L. Beguin, A. Signoles, T. Lahaye, A. Browaeys, G.-O. Reymond, and C. Jurczak, "Quantum computing with neutral atoms," *Quantum*, vol. 4, p. 327, Sept. 2020.

[9] S. Ebadi, A. Keesling, M. Cain, T. T. Wang, H. Levine, D. Bluvstein, G. Semeghini, A. Omran, J.-G. Liu, R. Samajdar, X.-Z. Luo, B. Nash, X. Gao, B. Barak, E. Farhi, S. Sachdev, N. Gemelke, L. Zhou, S. Choi, H. Pichler, S.-T. Wang, M. Greiner, V. Vuletic, and M. D. Lukin, "Quantum optimization of maximum independent set using rydberg atom arrays," *Science*, vol. 376, no. 6598, pp. 1209–1215, 2022.

[10] N. Coppola, R. Lescanne, L. Prost, and M. Žeško, "Think inside the box: Quantum computing with cat qubits," tech. rep., Alice & Bob, December 2024. Curated by Brian Siegelwax.

[11] C. Vercellino, P. Viviani, G. Vitali, A. Scionti, A. Scarabosio, O. Terzo, E. Giusto, and B. Montrucchio, "Neural-powered unit disk graph embedding: qubits connectivity for some qubo problems," in *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*, p. 186–196, IEEE, 2022.

[12] M. Nowak, B. Marchand, Y. Naghmouchi, S. Rainjonneau, W. Coelho, L. Vignoli, and L.-P. Henry, "Satellite mission planning with rydberg atoms," 2026.

[13] B. Korte and J. Vygen, *Combinatorial Optimization: Theory and Algorithms*. Springer, 5th ed., 2012.

[14] C. H. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*. Dover Publications, 1998.

[15] C. Berge, *The theory of graphs*. Courier Corporation, 2001.

[16] A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.

[17] B. N. Clark, C. J. Colbourn, and D. S. Johnson, "Unit disk graphs," *Discrete Mathematics*, vol. 86, no. 1, pp. 165–177, 1990.

[18] S. Tarquini, M. Vandelli, F. Ferrari, D. Dragoni, and F. Tudisco, "Drone delivery packing problem on a neutral-atom quantum computer," 2026.

[19] S. Tarquini, M. Vandelli, F. Ferrari, D. Dragoni, and F. Tudisco, "Emergency hub placement with a neutral-atom quantum computer," 2026.

[20] R. Nembrini, M. Ferrari Dacrema, and P. Cremonesi, "An application of reinforcement learning for minor embedding in quantum annealing," in *Proceedings of the 2nd International Workshop on AI for Quantum and Quantum for AI (AIQxQIA 2024) co-located with the 23rd International Conference of the Italian Association for Artificial Intelligence (AIxIA 2024)*, vol. 3913 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2024.

[21] R. Nembrini, M. Ferrari Dacrema, and P. Cremonesi, "Minor embedding for quantum annealing with reinforcement learning," *Quantum Machine Intelligence*, vol. 8, Feb. 2026.

[22] S. Das, S. K. Roy, R. Rana, and M. G. Chandra, "Grid-partitioned mwis solving with neutral atom quantum computing for qubo problems," *arXiv preprint arXiv:2510.18540*, 2025.

[23] F. Glover, G. Kochenberger, and Y. Du, "A tutorial on formulating and using qubo models," *arXiv preprint arXiv:1811.11538*, 2018.

[24] P. Date, R. Patton, C. Schuman, and T. Potok, "Efficiently embedding qubo problems on adiabatic quantum computers," *Quantum Information Processing*, vol. 18, no. 4, p. 117, 2019.

[25] Pasqal, "Pasqal delivers italy's first neutral atom quantum computer." `https://www.pasqal.com/newsroom/pasqal-delivers-italys-first-neutral-atom-quantum-computer/`, feb 2026. Accessed: 2026-07-22.

[26] G. Orsucci, "QUBO-GA-QUANTUM." `https://github.com/Giacomo-Orsucci/QUBO-GA-QUANTUM`, 2026. GitHub repository.

[27] A. Eiben and J. Smith, *Introduction To Evolutionary Computing*, vol. 45. 01 2003.

[28] C. Perron, Y. Bérubé-Lauzière, and V. Drouin-Touchette, "Leveraging analog neutral-atom quantum computers for diversified pricing in hybrid column-generation frameworks," *Physical Review A*, vol. 113, no. 2, p. 022617, 2026.

[29] Pasqal, "Qaa to solve a qubo problem | pasqal documentation." `https://docs.pasqal.com/pulser/tutorials/qubo/`, 2026. Accessed: 2026-07-27.

[30] Pasqal, "Qaa to solve a mwis problem - pasqal documentation." `https://docs.pasqal.com/pulser/tutorials/mwis/`, 2026. Accessed: 2026-07-27.